

MEDIHIVE

User Manual and Judge Quick Start

Public GitHub edition | ByteForge | Problem Statement PS02

A functional prototype for detecting medicine shortage risk and coordinating approved redistribution among a central warehouse, hospitals and pharmacies.

Quick start

- Clone or download the **MediHive-ByteForge** GitHub repository.
- Install Java 17 and Maven 3.9 or newer.
- Open the project folder containing **pom.xml** and **src/**.
- Run **mvn test**, then **mvn spring-boot:run**.
- Open **http://localhost:8080/login**. Do not open index.html directly.

Demo accounts

Facility	Username	Temporary password
Central Medical Warehouse	warehouse_admin	HiveWH!2026
Unity Hospital	unity_admin	UnityH!2026
City Hospital	city_hospital_admin	CityH!2026
CityCare Pharmacy	citycare_admin	CityCareP!2026
MediPlus Pharmacy	mediplus_admin	MediPlusP!2026

These fictional credentials are intentionally provided for local hackathon judging. They do not unlock a public production system. Never reuse them in a real deployment.

System map

The same Spring Boot server provides the website and REST API.

Layer	Main files	Responsibility
Browser UI	static/index.html, styles.css, app.js	Dashboard, dialogs, authenticated fetch calls and CSRF header.
Security	SecurityConfig, DemoAccounts, Access	Login, roles, facility ownership and authorization.
REST API	MediHiveController, ApiDtos	Catalogs, inventory, requests, offers and allocations.
Business logic	MediHiveService, StockPolicy	Risk, safe surplus, routing, approval, dispatch and receipt.
Persistence	JPA repositories and models	Facilities, medicines, inventory, requests and transfer history.
Database	H2 default; PostgreSQL profile	Resettable judge demo or persistent local database.

Control paths

Dashboard	Inventory changes
Quantity and daily use calculate days remaining and risk. Expiry within 60 days creates a warning.	Only the warehouse administrator can edit Central Medical Warehouse inventory. All hospital and pharmacy inventory is read-only.
Request routing	Transfer ledger
NORMAL/HIGH requests route to the warehouse. CRITICAL requests search ranked safe-surplus sources and may split the quantity.	PENDING -> APPROVED -> DISPATCHED -> DELIVERED. Dispatch subtracts source stock once; receipt adds destination stock once.

Complete request flow

Every changing action is rechecked by the backend.

Step	Actor and action	Result
1	Hospital or pharmacy creates a request.	Signed-in facility is fixed as requester.
2	Backend validates medicine, quantity, priority and ownership.	Invalid or unauthorized requests are rejected.
3A	NORMAL/HIGH priority.	A pending warehouse allocation is created.
3B	CRITICAL priority.	Safe sources are ranked; one or several pending offers may be proposed.
4	Source administrator approves an exact safe quantity or rejects.	Approved stock becomes reserved; total approvals cannot exceed the request.
5	Source administrator dispatches.	Safe stock is rechecked and source inventory is reduced once.
6	Receiving administrator confirms receipt.	Destination inventory increases once.
7	Backend totals delivered allocations.	Request closes only after the requested quantity is fully delivered.

A requester may cancel before dispatch or closure. Partial delivery remains visible with the remaining quantity. Duplicate dispatch or receipt is rejected.

Shortage calculations

- Days remaining = available quantity / average daily consumption. A zero-use item displays 999.
- Safety floor = max(reorder level, ceil(5 x average daily consumption)).
- Hospitals retain the safety floor plus 50 additional units; approved reservations are also protected.
- Expired inventory has zero transferable surplus.
- OUT_OF_STOCK: zero; CRITICAL: at/below reorder or at most 5 days; HIGH: at most 12 days; otherwise HEALTHY.

Setup and navigation

Use the included H2 database for the fastest judge demonstration.

Run from a terminal

```
mvn test
mvn spring-boot:run
```

Then open <http://localhost:8080/login>

Spring Tool Suite

- File > Import > Maven > Existing Maven Projects.
- Select the cloned MediHive-ByteForge folder containing pom.xml.
- Run MediHiveApplication.java as a Spring Boot App.

Main screen

Area	What it shows
Viewing facility	Warehouse can inspect all five facilities. Other accounts see only their assigned facility.
Dashboard	Tracked medicines, critical alerts, expiring stock and open requests.
Inventory	Quantity, days remaining, risk, expiry warning and safe surplus.
Requests	Requested quantity, remaining quantity, progress and source allocations.
Regional dispatches	Warehouse-only feed of dispatched and delivered transfers across all facilities.

PostgreSQL option

- Create a medihive database and set DB_URL, DB_USERNAME and DB_PASSWORD as environment variables.
- Run: **mvn spring-boot:run -Dspring-boot.run.profiles=postgres**.
- Never commit a real database password to GitHub.

Facility-specific permissions

Every local account controls exactly one facility.

Account	Can do	Cannot do
Warehouse	View all facilities and requests; edit warehouse inventory; approve and dispatch warehouse allocations; inspect regional dispatches.	Edit hospital/pharmacy stock; approve or dispatch another source's allocation; confirm another facility's receipt.
Hospital	View own hospital; create its requests; approve/reject/dispatch its outgoing offers; confirm its incoming deliveries.	Edit inventory; access another facility's controls.
Pharmacy	View own pharmacy; create its requests; approve/reject/dispatch its outgoing offers; confirm its incoming deliveries.	Edit inventory; access another facility's controls.

Before dispatch or receipt

- Check the medicine, source, destination and quantity.
- Dispatch subtracts stock from the source.
- Receipt confirmation adds stock to the requester.
- Refresh after switching users or reviewing another session.

Security boundary: hiding a button is not the protection. The backend validates role, facility, quantity and allocation state for every sensitive action.

Postman

- Import postman/MediHive.postman_collection.json.
- Use the logged-in JSESSIONID and XSRF-TOKEN cookies.
- For POST and PUT, copy the XSRF cookie value into the X-XSRF-TOKEN header.
- Replace example IDs with IDs returned by your running database.

Judge demonstration and submission

Recommended demonstration: the seeded 100-unit split request.

Step	Demonstration
1	Sign in as <code>unity_admin</code> and open Unity Hospital's seeded 100-unit CRITICAL adrenaline request.
2	Show the two proposals: 60 from the warehouse and 40 from City Hospital.
3	As <code>warehouse_admin</code> , approve 60 and dispatch. Show the warehouse stock decrease and regional feed.
4	As <code>city_hospital_admin</code> , approve and dispatch 40. Show that hospital inventory editing is blocked.
5	As <code>unity_admin</code> , confirm both receipts. Delivered reaches 100, remaining reaches 0 and the request closes.
6	Attempt a second receipt to demonstrate duplicate-action protection.

Troubleshooting

- Login denied: verify the exact demo username/password and confirm the server is running.
- Forbidden: verify the signed-in role and facility ownership.
- Approval/dispatch refused: reduce quantity, check surplus/expiry, or confirm the allocation state.
- Old styling: refresh with Ctrl+F5 after restarting the server.

Round 1 submission readiness

Item	Requirement
Presentation	Official template, PDF only, named with the actual Team ID, ByteForge and PS02.
Video	One English video; all registered members visible; public/unlisted accessible URL; maximum 3 minutes when demonstrating the prototype.
Prototype	Public GitHub repository accessible without requesting permission throughout judging.
Final check	Open every submitted link in an incognito window before the deadline.

Before presenting, run `mvn test` and complete the five-account browser walkthrough on the exact computer you will use.